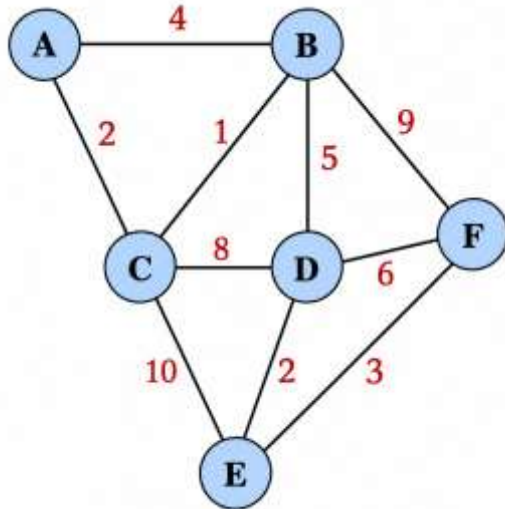


แบบฝึกหัด Dijkstra's Algorithm, Minimum Spanning Tree, Kruskal, Prim และ Borůvka

ข้อ 1: Dijkstra's Algorithm — Shortest Path

กำหนด Weighted Undirected Graph ดังนี้



กำหนด Edge ดังนี้

Edge	Weight
A-B	4
A-C	2
B-C	1
B-D	5
C-D	8
C-E	10
D-E	2
D-F	6
E-F	3
B-F	9

ให้ใช้ Dijkstra's Algorithm โดยเริ่มต้นที่ Vertex A

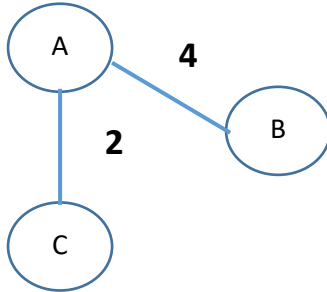
1.1 จงแสดงขั้นตอนของ Dijkstra ที่ละรอบ โดยระบุ

- Vertex ที่ถูกเลือก
- ค่า Distance ของ A, B, C, D, E และ F
- การ Relaxation ที่เกิดขึ้น

(Relaxation ใน Dijkstra's Algorithm คือกระบวนการ ตรวจสอบว่าเราพบเส้นทางที่สั้นกว่าเดิมไปยัง โหนดหนึ่งหรือไม่ ถ้าพบ ก็ปรับค่า Distance ของโหนดนั้นให้เล็กลง)

Relaxation = “ลองเส้นทางใหม่ แล้วถ้าสั้นกว่าเดิมก็ Update”

ตัวอย่าง สมมติว่ากำลังอยู่ที่โหนด A



ตอนเริ่มต้นกำหนด

$\text{Distance}(A) = 0$

$\text{Distance}(B) = \infty$

$\text{Distance}(C) = \infty$

เมื่อพิจารณาเส้นทาง $A \rightarrow B$

$\text{newDistance} = \text{Distance}(A) + \text{Weight}(A,B)$

$= 0 + 4$

$= 4$

เมื่อเปรียบเทียบกับค่าเดิมของ B เดิม $\text{Distance}(B) = \infty$ ใหม่ $= 4$

เพราะ $4 < \infty$ จึง Relax ค่า B เป็น $\text{Distance}(B) = 4$

สูตรหลักของ Relaxation คือ

$\text{newDistance} = \text{dist}[u] + \text{weight}(u,v)$

จากนั้นตรวจว่า

$\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$

ถ้าเป็นจริง ให้เปลี่ยน

$\text{dist}[v] = \text{dist}[u] + \text{weight}(u,v)$

ใน Java จะเขียนแนวคิดประมาณนี้:

```
int newDistance = distance[u] + weight;
```

```
if (newDistance < distance[v]) {
```

```
    distance[v] = newDistance;
```

```
}
```

จากข้อ 1 ให้ นักศึกษาเติมตาราง

รอบ	Vertex ที่เลือก	A	B	C	D	E	F
Initial	-	0	∞	∞	∞	∞	∞
1	A B=4,C=2	0	4	2	∞	∞	∞
2	C B=3,D=10,E=12	0	3	2	10	12	∞
3	B D=8,F=12	0	3	2	8	12	12
4	D E=10	0	3	2	8	10	12
5	E -	0	3	2	8	10	12
6	F -	0	3	2	8	10	12

1.2 จงหา Shortest Path จาก A ไปยัง

- D A-C-B-D, Total Weight = 8
- E A-C-B-D-E, Total Weight = 10
- F A-C-B-F, Total Weight = 12

พร้อมระบุ Total Weight

1.3 สมมติว่า Edge E-F ถูกเปลี่ยนจากน้ำหนัก 3 เป็น -5 อธิบายว่าเหตุใดจึงไม่ควรใช้ Dijkstra's Algorithm กับกราฟดังกล่าว

1.4 วิเคราะห์ Java Code

ตอบ: Dijkstra ใช้กับ Edge น้ำหนักติดลบไม่ได้ และ E-F=-5 ในกราฟแบบ Undirected ทำให้เกิดวงจรน้ำหนักติดลบ

```
for (Edge edge : graph[u]) {
    int v = edge.to;
    int weight = edge.weight;

    if (!visited[v] &&
        dist[u] + weight < dist[v]) {

        dist[v] = dist[u] + weight;
        parent[v] = u;
    }
}
```

ตอบคำถามต่อไปนี้

ก. เงื่อนไข **dist[u] + weight < dist[v]** มีความหมายอย่างไรใน Dijkstra?

ตรวจสอบว่าผ่าน u แล้วได้ระยะทางสั้นกว่าเดิมของ v หรือไม่

ข. **parent[v] = u** มีไว้เพื่ออะไร?

ใช้เก็บจุดก่อนหน้า เพื่อย้อนสร้าง Shortest Path

ค. หากลบเงื่อนไข **!visited[v]** ออก ผลกระทบที่อาจเกิดขึ้นคืออะไร?

ถ้าเราออก อาจแก้ dist ของจุดที่เลือกแล้ว ทำให้ Dijkstra ไม่ถูกต้อง

ข้อ 2: Kruskal's Algorithm

ใช้ Graph เดียวกับข้อ 1 จงหา Minimum Spanning Tree (MST) ด้วย Kruskal's Algorithm

2.1 เรียง Edge ทั้งหมดจาก Weight น้อยไปมาก

BC(1), DE(2), AC(2), EF(3), AB(4),
BD(5), DF(6), CD(8), BF(9), CE(10)

2.2 พิจารณา Edge ตามลำดับ และระบุว่า

- Accept — เลือกเข้า MST
- Reject — ไม่เลือก เพราะทำให้เกิด Cycle

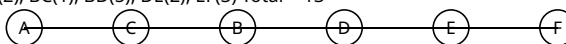
สร้างตาราง

ลำดับ	Edge	Weight	Accept/Reject	เหตุผล
1	BC	1	Accept	เชื่อม B-C ไม่เกิด Cycle
2	DE	2	Accept	เชื่อม D-E ไม่เกิด Cycle
3	AC	2	Accept	เชื่อม A กับ B-C ไม่เกิด Cycle
4	EF	3	Accept	ไม่เกิด Cycle
5	AB	4	Reject	เกิด Cycle
6	BD	5	Accept	เชื่อม 2 Component

หยุดเมื่อ MST มีจำนวน Edge ครบตามที่ต้องการ

MST = AC(2), BC(1), BD(5), DE(2), EF(3) Total = 13

2.3 วาด MST ที่ได้ และหาหน้าหนักรวม



2.4 ถ้ากราฟมี V Vertices จำนวน Edge ใน MST ต้องเป็นเท่าใด และเพราะเหตุใด?

2.5 วิเคราะห์ Java Code

```

for (Edge edge : edges) {

    int rootU = find(edge.u);
    int rootV = find(edge.v);

    if (rootU != rootV) {
        mst.add(edge);
        union(rootU, rootV);
    }
}
  
```

ก. อธิบายว่า **rootU != rootV** ช่วยป้องกันปัญหาอะไร?

ข. และถ้า **rootU == rootV** แล้วเรายังเพิ่ม Edge เข้า MST จะเกิดอะไรขึ้น?

2.4 ตอบ: $V-1 = 5$ Edges เพราะ MST ต้องเชื่อมทุก Vertex โดยไม่เกิด Cycle

2.5 ก. ป้องกันการเพิ่ม Edge ที่ทำให้เกิด Cycle ใน MST

2.5 ข. จะเกิด Cycle และ MST จะไม่ถูกต้อง

ข้อ 3: Prim's Algorithm และการเปรียบเทียบกับ Kruskal

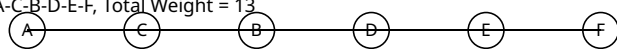
ใช้ Graph เดียวกับข้อ 1 ให้หา MST ด้วย Prim's Algorithm โดยเริ่มจาก Vertex A

3.1 แสดง Edge ที่ Prim เลือกในแต่ละรอบ

รอบ	Vertices ใน MST	Candidate Edges ที่สำคัญ	Edge ที่เลือก	Weight
1	A	AB(4), AC(2)	AC	2
2	A,C	AB(4), BC(1), CD(8), CE(10)	BC	1
3	A,C,B	BD(5), BF(9), CD(8), CE(10)	BD	5
4	A,C,B,D	DE(2), DF(6), BF(9), CE(10)	DE	2
5	A,C,B,D,E	EF(3), DF(6), BF(9), CE(10)	EF	3

3.2 วาด MST ที่ได้และหาน้ำหนักรวม

MST = A-C-B-D-E-F, Total Weight = 13



3.3 เปรียบเทียบแนวคิดของ Prim กับ Kruskal โดยอธิบายว่า

- Algorithm ใดเลือก Edge ที่เล็กที่สุดของทั้ง Graph
- Algorithm ใดเลือก Edge ที่เล็กที่สุดที่เชื่อม Tree ปัจจุบันกับ Vertex ภายนอก

Kruskal

Prim

3.4 นักศึกษาคนหนึ่งกล่าวว่า

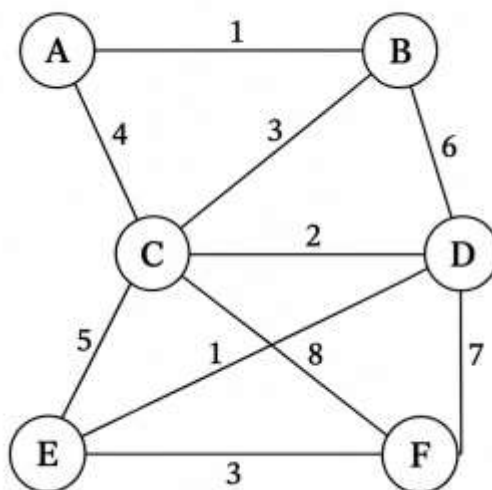
“Prim และ Dijkstra เหมือนกัน เพราะทั้งคู่เลือกค่าที่น้อยที่สุดในแต่ละรอบ ดังนั้นผลลัพธ์ต้องเป็น Tree เดียวกัน”

จงวิเคราะห์ว่าข้อความนี้ถูกหรือผิด พร้อมอธิบายเหตุผล

ตอบ: ผิด เพราะ Prim ใช้หา MST ส่วน Dijkstra ใช้หา Shortest Path ดังนั้นเกณฑ์เลือกและผลลัพธ์ไม่จำเป็นต้องเป็น Tree เดียวกัน

ข้อ 4: Boruvka's Algorithm

Question 4: Boruvka's Algorithm



เริ่มต้นให้แต่ละ Vertex เป็น Component ของตนเอง {A} {B} {C} {D} {E} {F}

4.1 ใน Borůvka รอบที่ 1 ให้หา Cheapest Outgoing Edge ของแต่ละ Component

Component	Cheapest Edge	Weight
{A}	AB	1
{B}	AB	1
{C}	CD	2
{D}	DE	1
{E}	DE	1
{F}	EF	3

จากนั้นระบุ Components ใหม่หลังจาก Merge

=> {A,B}, {C,D,E,F}

4.2 ดำเนินการ Borůvka รอบที่ 2 จนได้ MST

ให้ระบุ

- Components ก่อนเริ่มรอบ

{A,B}, {C,D,E,F}

- Cheapest outgoing edge ของแต่ละ Component

{A,B}->BC(3), {C,D,E,F}->BC(3)

- Edge ที่ถูกเพิ่มเข้า MST

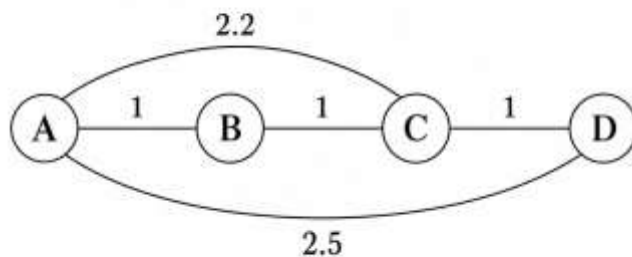
BC(3) -> {A,B,C,D,E,F}

ข้อ 5: วิเคราะห์ Shortest Path vs Minimum Spanning Tree

มหาวิทยาลัยต้องการเชื่อมต่ออาคาร 4 อาคารด้วย Network Cable

กำหนด Graph

Question 5: Shortest Path vs MST



สถานการณ์ A

มหาวิทยาลัยต้องการเชื่อมต่อทุกอาคารเข้าด้วยกันโดยใช้สาย Network รวมให้น้อยที่สุด

5.1 ควรใช้ Algorithm ประเภทใดระหว่าง Dijkstra หรือ MST Algorithm พร้อมอธิบายเหตุผล

5.2 หา MST และคำนวณ Total Weight

ตอบ 5.1: ใช้ MST เพราะต้องเชื่อมต่อทุกอาคารและให้สาย Network รวมสั้นที่สุด

ตอบ 5.2: MST = AB(1), BC(1), CD(1) Total Weight = 3

สถานการณ์ B

กำหนดให้ A เป็น Server หลัก ต้องการหา เส้นทางจาก A ไป D ที่มี Cost ต่ำที่สุด

5.3 ใช้ Dijkstra หรือการวิเคราะห์ Shortest Path หาเส้นทางที่สั้นที่สุดจาก A ไป D A→D asy, Cost=2.5

5.4 ถ้าใช้เฉพาะ Edge ใน MST จากข้อ 5.2 เส้นทางจาก A ไป D มี Cost เท่าใด?

เปรียบเทียบกับ Shortest Path จากข้อ 5.3 MST path: A-B-C-D = 1+1+1 = 3 มากกว่า Shortest Path = 2.5

5.5 จากผลที่ได้ จงวิเคราะห์ข้อความต่อไปนี้ “Minimum Spanning Tree ทำให้เส้นทางจาก Source ไปทุก Vertex สั้นที่สุดเสมอ” ถูกหรือผิด เพราะเหตุใด?

ตอบ 5.5: ผิด เพราะ MST ทำให้ผลรวมของ Edge ในต้นไม้มีน้อยที่สุด แต่ไม่ได้ทำให้เส้นทางจาก Source ไปทุก Vertex สั้นที่สุด